

黄渡理工大学 × 同济大学

百年
修得同船渡
未来也要一起走

实验报告名称: 《高级语言程序设计》课程大作业1——汉诺塔. 实验报告

班级: 5000244001602

学号: 2452098

姓名: 赵崇治

完成日期: 2025年5月17日

1. 汉诺塔问题以及相关背景

1.1. 题目

题目主要可以分为两个部分，一是整合以往的4个小作业实现前4个菜单(基本解)，二是使用cmd下的伪图形工具实现后5个菜单(图形解)。其中只有最后2个菜单项是最终产品，其余均为副产品(或中间产物)。

1.2. 背景

汉诺塔 (Tower of Hanoi)，又称河内塔，原是一个源于印度古老传说的益智玩具。大梵天创造世界的时候做了三根金刚石柱子，并且规定，在小圆盘上不能放大圆盘，在三根柱子之间一次只能移动一个圆盘。在19世纪，数学家们研究了这个问题，并找到了解决方法。这让汉诺塔问题变成了著名的计算机科学问题之一，哪怕在现代看来这只是一个多么简单的问题。

假设有 n 个圆盘，移动次数是 $f(n)$ 。 $f(1)=1, f(2)=3, f(3)=7$ ，且 $f(k+1)=2*f(k)+1$ 。此后不难证明 $f(n)=2^n-1$ 。 $n=64$ 时，假如每秒钟一次(传说中)，共需多长时间呢？一个平年365天有31536000 秒，闰年366天有31622400秒，平均每年31556952秒，计算一下：18446744073709551615秒
这表明移完这些金片需要5845.54亿年以上，而地球存在至今不过45亿年，太阳系的预期寿命据说也就是数百亿年。真的过了5845.54亿年，不说太阳系和银河系，至少地球上的一切生命，连同梵塔、庙宇等，都早已经灰飞烟灭。

庆幸的是，现代人类社会拥有计算机，不需要计算到天荒地老。

1.3. 解决

经过思考会发现，不管过程如何曲折，想要把A上 n 个盘子移到C上，就必须要把前 $n-1$ 个盘子先放到B上，然后将最大的盘子放到C上；之后再将C上的固定了的盘子假设不存在，此刻B上的盘子当做上一步的A盘子，继续一样的思路进行。

1.4. 模拟

使用栈来模拟这一移动过程是再合适不过了，因为这一虚拟结构和真实物理结构同样具有 First In Last Out 的特点，此外，在使用数组实现栈时还可以用一个整数指向栈顶的位置，便于操作。

2. 整体设计思路

程序主要封装为3个部分。首先通过main函数入口进入菜单(menu)函数执行菜单的选择，能在结束后按下回车(各种交互)重复选择菜单(直到选择0.退出菜单)，这是第一部分。

然后菜单(op)获取完毕后，根据情况选择进入解决(solve)函数，这是一个可以通过不同的op进入调用不同的解决方案的指挥函数，使得衔接井井有条。接着按照理想情况解决所有内部运算，这是第二部分。

在进入问题解决时，各个部分的solve函数会调用各种output函数，且通过参数op的不同(在必要情况下还可以增设多一个order参数)输出各种语句(图形或文字)，这是用户能感知到的，是程序的第三部分。

3. 主要功能的实现

1. menu: 显示各菜单项，读入正确的选项后返回值op。

2. input: 选项的集散地，通过该函数可以直接执行5/6简单操作菜单，其余进入次级函数解决。

3. void manageError(int* n, char* src, char* tmp, char* dst, int op):

使用函数形参为实参指针的方法同时改变多个实参值，为后续调用提供数值传入。

在实现时可以复用先前小作业代码

4. init: 处理完输入后的准备工作(基本解)，为基本的打印前处理内部数组。

在实现时可以复用先前小作业代码。

5. initprint: 处理完输入后的准备工作(图形解)，值得一提的是为了不写死菜单出现位置，这里无法复用以及先前函数大致无法复用小作业的代码，可惜的是先前小作业写死了(x, y)的位置。

这里会调用打印柱子(图形解)的函数作为初始打印。

此外，这里需要传入op参数，这样可以将6/7/8/9菜单的初始图形打印都整合进来。

因为6选项较为简单，initprint直接实现了6的输出部分。

6. hanoi: 由于整个函数只允许只用1个递归函数，因此为了精简本递归函数的代码，相比与先前的小作业，本函数增加了op参数的传入，顺应了题目的需求，然后将op传递到solve函数中，迁移分担了hanoi的主体功能，使得hanoi成为“形式递归”就像英语中的“形式主语”。

这也意味着先前小作业的递归函数很大程度上需要推翻重写。

7. solve: 这里存放的是大部分菜单的解决方案, 从hanoi中迁出, 根据op的不同, 进一步外延出不同的solve解决方案, 以分担任务, op==4时, 笔者观察到输出部分大体相同, 于是整合为了一个Output_4函数。不过op==8时并没有这么做, 感觉可有可无就没做了。这样并没有使得可维护性略微增强。

8. outputGraph: 本意想“复用”5-b7的代码 用于输出纵向数组, 也模拟了竖着的柱子。这里同样需要传入op参数, 这样可以供4/8/9菜单同时使用

同样, 值得一提的是为了不写死柱子出现位置, 这里“无法”复用以及先前函数, 大致无法复用小作业的代码, 可惜的是先前小作业写死了(x, y)的位置。

该函数内部为了得到位置, 需要进行一些数学的坐标简单加减运算。

9. OutputStatus: “复用”, 5-b7代码, 用于横向地打印内部显示数组。

实际上该函数并不能完全复用先前小作业, 笔者在先前函数内部加了坐标定位, 但同样碍于const_value的存在, 删除了不必要的cct_gotoxy并重新整理出去了相关定位使得op不必再传入。

10. Move: 所有需要用到递归函数的函数的共用部分, 因为要操作内部数组就必须实现内存上真正的“移动”。在本周习题课上, 沈老师指出移动顶盘后, 无需将原位置置零, 因为在下一个盘子移入以前指针不会再指向先前位置, 因此这是安全的, 抹零是冗余操作。笔者做的时候没想到, 但是没有对此进行修改, 毕竟我确实没想到就不写上去了。

11. animate: 7/8/9盘子移动的动画在该函数中给出。相比与传统方案, 本次大作业使用了hdc_tools, 可以绘制有颜色的矩形, 而不是使用带颜色的‘ ’ (space)填充, 这使得整体呈现效果变得更好了, 并且这对坐标的控制、函数的实现也提出了更高的要求?

本函数分为了3个部分, 分别是向上移动、向左/右移动, 向下移动, 三个循环。

在实现时有几个需要注意的点。

一是控制台原点的坐标以左上角为原点, 向右向下增大。而这与人的常识有点相悖, 需要编写一段时间后笔者才适应。

二是const_value中给出的top_y指的是视觉意义上的top_y坐标, 在循环中, 向上的终止条件需要写为>=top_y, 向上-=step_y, 向下同理。

三是在抹除原像素点时, 需要注意分为两部分(分区间段), 需要再离开柱顶以前补全原柱颜色, 操作时加一个条件判断即可。

四是不能加入cls()等清屏, 不能再绘图中跨越光标, 使得文字/图形被覆盖。

12. solve_9: 实现菜单9的游戏部分

初始化

函数

<code>manageError(&n, &src, &tmp, &dst)</code>	输入的层数(1-10)、起始柱/目标柱(A-C)，并自动计算中转柱
<code>init(9, n, src)</code>	初始化全局数组 <code>stk</code> 和 <code>ptr</code> ，将初始盘子放 <code>src</code> 柱
<code>initPrint()</code>	清屏并绘制初始图形界面（柱子+盘子）

函数

作用

<code>Move(char from, to)</code>	修改 <code>stk</code> 和 <code>ptr</code> 数组，实现盘子移动的逻辑
<code>animate()</code>	通过 <code>hdc_rectangle</code> 绘制移动动画
<code>outputGraph(9)</code>	在控制台用字符画显示当前三根柱子的状态
<code>wait()</code>	根据 <code>speed</code> 参数控制动画速度（0=按回车继续，1=1ms 延时）

与其他菜单的差异

特性	手动	自动演示
控制方式	用户实时输入 AC/Q	程序自动递归求解
移动验证	需要检测大盘压小盘规则	程序保证合法性
显示需求	实时响应操作+动画	按步骤刷新界面
结束条件	用户主动退出或胜利	递归自然结束

大体上来说，其实菜单9的实现思路是非常清晰的，值得一提的是在输入ac/abbaababa/q时，需要琢磨一下如何达到demo的效果/比demo获取输入做得更好。

在输入移动命令时，笔者使用 `_getch()` 以最大程度地避免错误输入的处理。并且笔者对q的退出输入进行了额外处理，确保不会每一种错误输入/正确输入都有对应的情况给予用户提示。但笔者并没能很好地精简压缩这一部分函数的行数，顺其自然就好。

4. 调试过程碰到的问题

碰到的问题主要是归结于心态的不平稳。好吧，有时候这种东西就很吃心态，一些很朴素的东西由于脑子一抽想错了，或者数学关系算炸毛了，或是要求过于“繁琐”导致情绪出现了一点起伏。解决方法是：睡一觉/吃饭/打两把游戏就好了。调节一下情绪，这也是完成作业的一部分。

只要静下心来阅读pdf，反复阅读，在本人编写程序过程中已经解决了99%的问题。

起初在整理前面的4项菜单时，心态是非常炸裂的，我很难想象我还要重新整合一遍这些小作业，居然要把递归函数推翻，然后全部输出踢出去。我认为这个曲线是有点陡峭的，没有什么提示。实际上在递归函数中不限制那么死行数依然可以做到较好的工程维护性。

哪怕这改起来不是什么难事，但依然是一件麻烦事，笔者在大作业完成过程中从来不把难事和麻烦事画上等号，尽管再麻烦，这依旧不是一个难的作业。造火箭麻烦，但造火箭更难。

在调试盘子输出位置时，一开始盘子总是穿出底部，或者是放到顶上悬空，我反复调试后发现是指针指向的位置和盘子实际落下的位置不同，但内部数组的移动和盘子的移动不是同时的，这就导致笔者又需要细细静下心来推导其中关系，修改实现逻辑。

5. 心得体会

5.1 心得体会

在完成一些较为复杂的题目时，分为若干道小题比较好，但像高精度幂和 xx 行的行数限制这样的还是太突兀，学习曲线似乎不太平滑。

从先前的小作业来看，能够借鉴的实际不多。更多的还是推翻了重新组合，不像空间站有固定的接口，你是什么航天器就能接，不是什么航天器就不能接。很多前面已经实现过的功能可能需要和别的相似的功能重新提取整理出去，然后又将新的写回去。如果在前面就这么做(每个功能都划分出一个特定函数)显然是不可能的，难道我还能未卜先知不成，更何况 demo 的实现哪怕是整合先前的作业，实际的输出细节依旧和先前作业有差别。

如果非要为了未卜先知，未知的潜在的重用代码而操作实现一些细微的函数，也许是舍本逐末的。当然这只是笔者目前所执一面之词，日后回顾时定会有更深刻的认识。

在项目初期，或许会为了快速实现某个功能（例如界面元素的定位 `MenuItemX_Start_X`，或是动画中的 `HDC_Base_Width`），不自觉地就在代码中直接使用了数值。当需求变更，比如需要调整界面布局，或是适配不同大小的盘子时，这些散落在各处的硬编码值就成了噩梦的开始——逐一查找、修改，不仅耗时耗力，还极易引入新的错误。本次项目中，有意识地将这些值提取到如 `hanoi_const_value.h` 这样的常量定义文件中，或通过参数（如 `op`）动态计算坐标，虽然增加了初期的思考成本，但其后期的维护便利性不言而喻。这让我深刻体会到，养成良好编程习惯，需要从每一行代码做起，从设计之初就具备前瞻性，主动规避硬编码，用常量、枚举、配置文件或动态计算来提升代码的“柔性”。

在传递参数时，笔者有意地往往都会不管三七二十一传入一个选项(`op`)以及层数(`n`)，笔者大概感觉这是一个不算坏的习惯，

常云“不要写死数值”，道理都懂，实际习惯的养成并非一朝一夕之事。通过一门课程可以习得这些概念，但大学四年能有多少机会参与到工程项目中？纸上得来终觉浅，绝知此事要躬行。光写不是真功夫，真得。

本次汉诺塔项目，虽然本质上仍是一个课程作业，但其涉及的模块之多（菜单、输入处理、核心逻辑、多种模式的输出与动画）、对用户体验的追求（如动画效果、错误提示），已然让我初步感受到了小型项目开发的完整流程。当理论知识（如递归、数组模拟栈）应用到具体场景，并需要与其他模块（如图形绘制、用户交互）协同工作时，才真正暴露出“纸上得来终觉浅”的现实。例如，`hanoi` 函数本身递归逻辑不难，但如何将其与不同的 `op`（操作选项）结合，优雅地分发到 `solve` 函数，并进一步细化到 `Output_4` 这样的具体输出模块，就需要超越单纯算法实现的工程思维。这让我认识到，要缩短理论与实践的距离，唯有积极寻找并投身于更复杂的实践项目中，哪怕是课程设计，也应以前工程项目的标准来要求自己。

“光写不是真功夫”，对此我深有同感。真正的“功夫”并不仅仅是让代码运行起来，更在于如何写出结构清晰、逻辑严谨、易读易维护的代码。这包括了良好的问题分解能力（将复杂问题拆解为可管理的模块，独立函数），选择合适的数据结构和算法，以及在遇到困难时系统性的调试和解决问题的能力。

5.2 教训

下次还敢写死通过本次汉诺塔项目，我对“避免硬编码”的理解从“知道”深化到了“体会”，更对理论联系实际的重要性有了切身的感受。编程不仅仅是技术的堆砌，更是一种思维的艺术和工程的实践。认识到心态调节也是完成作业的一部分，这同样是宝贵的成长。

未来的学习和实践中，我定当更加警惕硬编码的诱惑，主动运用常量、配置等手段提升代码质量。同时，也会更积极地寻求实践机会，哪怕是新项目，也力求从设计、实现到调试的完整体验，不断“躬行”，以求“绝知”。这一次的“教训”——如果再“写死”，定要更深刻地反思并寻求更优的方案。

6. 附件：源程序(分两栏)

注意排版格式，对齐方式，可以采用两列排版，正文采用宋体，小五，行距为单倍行距

不需要全部源程序，main函数、菜单选择、输入输出错误等常规内容建议舍弃，挑部分核心代码贴上来即可。

```
void solve_4(int op)
{
    int n;
    char src;
    char tmp;
    char dst;
    manageError(&n, &src, &tmp, &dst, op);
    while (true) {
        cout << "请输入移动速度(0-200: 0-按
回车单步演示 1-200:延时 1-200ms)\n";
        cin >> speed;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(2147483647, '\n');
            continue;
        }
        else {
            if (speed <= 200 && speed >= 0)
                break;
        }
    }
    isShow = 1;
    cct_cls();
    cout << "从 " << src << " 移动到 " << dst
<< ", 共 " << n << " 层, 延时设置为 " << speed <<
"(ms)";
    init(4, n, src);
    hanoi(n, src, tmp, dst, op);
    outputGraph(4);
    cct_gotoxy(0, MenuItem4_Start_Y + 2);
}
```

```
void solve_6(int op)
{
    int n;
    char src;
    char tmp;
    char dst;
    manageError(&n, &src, &tmp, &dst, op);
    initPrint(op, steps, n, src, dst);
}

void solve_7(int op)
{
    int n;
    char src;
    char tmp;
    char dst;
    manageError(&n, &src, &tmp, &dst, op);
    while (true) {
        cout << "请输入移动速度(0-1: 0-按回
车单步演示 1:延时 1ms)";
        cin >> speed;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(2147483647, '\n');
            continue;
        }
        else {
            if (speed == 0 || speed == 1)
                break;
        }
    }
    initPrint(op, steps, n, src, dst);
    init(7, n, src);
    hanoi(n, src, tmp, dst, 7);
}
```



```

void solve_8(int op)
{
    int n;
    char src;
    char tmp;
    char dst;
    manageError(&n, &src, &tmp, &dst, op);
    while (true) {
        cout << "请输入移动速度(0-1: 0-按回
车单步演示 1:延时 1ms)";
        cin >> speed;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(2147483647, '\n');
            continue;
        }
        else {
            if (speed == 0 || speed == 1)
                break;
        }
    }
    cct_cls();
    hdc_cls();
    init(8, n, src);
    initPrint(8, steps, n, src, dst);
    hanoi(n, src, tmp, dst, 8);
}

void solve_9(int op)
{
    int n;
    char src;
    char tmp;
    char dst;
    manageError(&n, &src, &tmp, &dst, op);
    while (true) {
        cout << "请输入移动速度(0-1: 0-按回
车单步演示 1:延时 1ms)\n";
        cin >> speed;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(2147483647, '\n');
            continue;
        }
        else {

```

```

            if (speed == 0 || speed == 1)
                break;
        }
    }
    cct_cls();
    hdc_cls();
    init(9, n, src);
    initPrint(9, steps, n, src, dst);
    outputGraph(9);
    int step = 0;
    while (true) {
        char from, to;
        if (ptr[dst - 'A'] == n) {
            cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y + 5);
            cout << "我嘞个豆, 这你都能通关,
不赖! ";
            break;
        }
        cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y + 2);
        cout << "请输入移动的柱号(命令形式:
AC=A 顶端的盘子移动到 C, Q=退出):
";

        cct_gotoxy(MenuItem9_Start_X + 59,
MenuItem9_Start_Y + 2);
        int ch;
        while (true) {
            ch = _getch();
            if (ch >= 'a' && ch <= 'c' || ch >=
'A' && ch <= 'C' || ch == 'q' || ch == 'Q') {
                from = ch;
                cout << from;
                break;
            }
        }
        while (true) {
            ch = _getch();
            if (ch >= 'a' && ch <= 'c' || ch >=
'A' && ch <= 'C') {
                to = ch;
                cout << to;
                break;
            }
        }
        if (ch == 13)
            break;
    }
}

```

```

    }
    if (ch == 13) {
        if (from == 'q' || from == 'Q') {

            cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y + 5);
            cout << "游戏结束! ";
            break;
        }
        else
            continue;
    }
    else if (from == 'q' || from == 'Q')
        continue;
    if (from >= 'a' && from <= 'c')
        from -= 32;
    if (to >= 'a' && to <= 'c')
        to -= 32;
    if (from < 'A' || from > 'C' || to < 'A' ||
to > 'C') {
        cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y + 4);
        cout << "输入错误, 柱号必须是 A、
B、C 之一! ";
        Sleep(2000);
        cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y + 4);
        cout << "
";
        continue;
    }
    if (from == to) {
        cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y + 4);
        cout << "源柱和目标柱不能相同!
";
        Sleep(2000);
        cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y + 4);
        cout << "
";
        continue;
    }
    if (ptr[from - 'A'] == 0) {
        cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y + 4);
        cout << "源柱为空, 无法移动!
";
        Sleep(2000);
        cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y + 4);
        cout << "
";
        continue;
    }
    if (ptr[to - 'A'] > 0 && stk[from -
'A'] > 0 && ptr[from - 'A'] > ptr[to - 'A'] - 1)
    {
        cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y + 4);
        cout << "不能大盘压小盘, 请重输!
";
        Sleep(2000);
        cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y + 4);
        cout << "
";
        continue;
    }
    step++;
    int disk_num = 0;
    cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y);
    cout << "第" << right << setw(4) <<
step << " 步(" << setw(2) << stk[from - 'A'] << ptr[from
- 'A'] - 1 << " #: " << from << "-->" << to << ") ";
    disk_num = stk[from - 'A'] << ptr[from -
'A'] - 1;
    Move(from, to);
    outputGraph(9);
    cct_gotoxy(MenuItem9_Start_X,
MenuItem9_Start_Y);
    outputStatus(9);
    animate(disk_num, from, to, 9,
ptr[from - 'A'], ptr[to - 'A'] - 1);
    wait();
}
}

```

```

void animate(int n, char src, char dst, int op, int
from_top, int to_top)
{
    const int dist = dst - src;
    int srcX[3];
    for (int i = 0; i < 3; i++)
        srcX[i] = HDC_Start_X + (23 * i + 11) *
HDC_Base_Width + HDC_Underpan_Distance * i;
    int width = (2 * n + 1) * HDC_Base_Width;
    int color = HDC_COLOR[n];
    if (steps < 7)
        wait();
    for (int y = HDC_Start_Y - (from_top + 1) *
HDC_Base_High - HDC_Step_Y; y >= HDC_Top_Y; y
-= HDC_Step_Y) {
        if (y < HDC_Start_Y - from_top *
HDC_Base_High) {
            hdc_rectangle(srcX[src - 'A'] -
width / 2 + HDC_Base_Width / 2,
y + HDC_Base_High,
width, HDC_Step_Y,
HDC_COLOR[0]);
            if (y + HDC_Base_High >=
HDC_Start_Y - 12 * HDC_Base_High) {
                hdc_rectangle(srcX[src - 'A'],
y + HDC_Base_High,
HDC_Base_Width,
HDC_Step_Y,
HDC_COLOR[11]); // 如
果清除区域与柱子重叠, 重绘柱子部分
            }
        }
        hdc_rectangle(srcX[src - 'A'] - width / 2
+ HDC_Base_Width / 2,
y,
width, HDC_Base_High,
color);
        if (steps < 7)
            wait();
    }
    if (dist > 0)
        for (int x = srcX[src - 'A']; x <= srcX[dst
- 'A']; x += HDC_Step_X) {
            if (x > srcX[src - 'A']) {
                hdc_rectangle(x -
HDC_Step_X - width / 2 + HDC_Base_Width / 2,
HDC_Top_Y,
HDC_Step_X,
HDC_Base_High,
HDC_COLOR[0]);
            }
            hdc_rectangle(x - width / 2 +
HDC_Base_Width / 2,
HDC_Top_Y,
width, HDC_Base_High,
color);
            if (steps < 7)
                wait();
        }
    else if (dist < 0)

```

```

        for (int x = srcX[src - 'A']; x >= srcX[dst
- 'A']; x -= HDC_Step_X) {
            if (x < srcX[src - 'A']) {
                hdc_rectangle(x + width / 2 +
HDC_Base_Width / 2,
HDC_Top_Y,
HDC_Step_X,
HDC_COLOR[0]);
            }
            hdc_rectangle(x - width / 2 +
HDC_Base_Width / 2,
HDC_Top_Y,
width, HDC_Base_High,
color);
            if (steps < 7)
                wait();
        }
    }
    for (int y = HDC_Top_Y; y <= HDC_Start_Y -
(to_top + 1) * HDC_Base_High; y += HDC_Step_Y) {
        if (y > HDC_Top_Y) {
            hdc_rectangle(srcX[dst - 'A'] -
width / 2 + HDC_Base_Width / 2,
y - HDC_Step_Y,
width, HDC_Step_Y,
HDC_COLOR[0]);
            if (y - HDC_Step_Y >=
HDC_Start_Y - 12 * HDC_Base_High) {
                hdc_rectangle(srcX[dst - 'A'],
y - HDC_Step_Y,
HDC_Base_Width,
HDC_Step_Y,
HDC_COLOR[11]); // 如
果清除区域与柱子重叠, 重绘柱子部分
            }
        }
        hdc_rectangle(srcX[dst - 'A'] - width / 2
+ HDC_Base_Width / 2,
y,
width, HDC_Base_High,
color);
        if (steps < 7)
            wait();
    }
}

void hanoi(int n, char src, char tmp, char dst, int
op)
{
    if (n == 1) {
        steps++;
        return solve(n, src, dst, op, 1);
    }
    hanoi(n - 1, src, dst, tmp, op);
    steps++;
    solve(n, src, dst, op, 2);
    if (op != 7)
        hanoi(n - 1, tmp, src, dst, op);
}

```

```

void solve_8(int op)
{
    int n;
    char src;
    char tmp;
    char dst;
    manageError(&n, &src, &tmp, &dst, op);
    while (true) {
        cout << "请输入移动速度(0-1: 0-按回车
单步演示 1:延时 1ms)";
        cin >> speed;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(2147483647, '\n');
            continue;
        }
        else {
            if (speed == 0 || speed == 1)
                break;
        }
    }
    cct_cls();
    hdc_cls();
    init(8, n, src);
    initPrint(8, steps, n, src, dst);
    hanoi(n, src, tmp, dst, 8);
}
    
```